

From Heuristics to Graph Neural Networks: Benchmarking Modularity Maximization for Community Detection

Adil Chhabra
adil.chhabra@informatik.uni-
heidelberg.de
Heidelberg University
Heidelberg, Germany

Kenneth Langedal*
kenneth.langedal@informatik.uni-
heidelberg.de
Heidelberg University
Heidelberg, Germany

Christian Schulz
christian.schulz@informatik.uni-
heidelberg.de
Heidelberg University
Heidelberg, Germany

Abstract

Community detection methods commonly optimize modularity, an objective function designed to maximize intra-community connectivity while penalizing inter-community connectivity relative to random expectation. Over the past two decades, a wide range of algorithms have been proposed to optimize modularity, spanning classical heuristic and exact methods as well as more recent machine learning-based approaches that incorporate node attributes. Yet, survey literature and experimental evaluations remain fragmented, often confined to narrow subsets of algorithms or a single paradigm, limiting insight into relative strengths and trade-offs in practice. In this work, we present the first systematic experimental survey of modularity-based graph clustering algorithms that jointly evaluates classical and learning-based approaches. We study 17 representative algorithms spanning greedy, multilevel, metaheuristic, streaming, exact, and neural paradigms, and organize them within a unified taxonomy grounded in their algorithmic design principles. Through a consistent experimental evaluation across multiple datasets and evaluation criteria, we analyze empirical trade-offs in solution quality and scalability, providing comparative insight rather than identifying a single best method. We additionally contribute a unified, open-source framework for interfacing all 17 evaluated clustering algorithms, supporting reproducibility and future benchmarking.

CCS Concepts

• **Theory of computation** → **Graph algorithms analysis**; • **Mathematics of computing** → **Graph theory**; • **Computing methodologies** → **Cluster analysis**.

Keywords

Community detection, Graph clustering, Modularity optimization, Experimental evaluation

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

KDD '26, Jeju, Korea

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

ACM Reference Format:

Adil Chhabra, Kenneth Langedal, and Christian Schulz. 2026. From Heuristics to Graph Neural Networks: Benchmarking Modularity Maximization for Community Detection. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26)*. ACM, New York, NY, USA, ?? pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Networks are useful in modeling complex systems across the natural sciences, social sciences, and engineering. Such systems are represented as graphs, where vertices denote entities and edges encode interactions between them, as in social networks, communication infrastructures, biological and financial systems, and information networks. A common feature of real-world graphs is *community structure*, where vertices form groups with dense internal connectivity and sparser external connections.

Identifying these groups, commonly termed *graph clustering* or *community detection*, is a central task in network analysis, with applications ranging from social and political analysis and fraud detection [?] to bioinformatics [?], program optimization [?], and epidemic modeling [?]. Beyond application-specific insights, community structure reveals fundamental organizational patterns, such as cohesive subgraphs, boundary vertices, and inter-community interactions. Despite extensive study, no single clustering method performs best across all network types [?], motivating continued algorithmic development and comparative evaluation.

Most community-detection methods formulate clustering as an optimization problem over vertex partitions, assigning a quality score to each candidate solution. Among clustering objectives, *modularity* [?], which measures the density of edges inside communities as compared to edges between communities while accounting for random chance, is widely adopted and shown to be effective [?]. Thus, this survey focuses on modularity-based methods, while including widely used non-modularity baselines.

We distinguish two broad classes of modularity-based algorithms. *Classical* methods rely on discrete optimization heuristics driven solely by graph topology and a well-defined objective function, yielding interpretable optimization behavior. In contrast, *learning-based* approaches integrate modularity into deep learning pipelines, most commonly via graph neural networks, incorporating node attributes alongside graph topology through feature representations. This added expressiveness introduces trade-offs in interpretability, training complexity, and computational cost [?].

Community detection faces challenges arising from the scale, complexity and frequently-updating content of real-world graphs. Effective methods must balance solution quality, often with noisy

data lacking ground truth, with runtime and memory scalability. Sensitivity to parameters, such as assumed cluster counts, further complicates practical deployment [?]. Understanding how different algorithms navigate these trade-offs remains an open challenge.

Despite existing surveys [?????], there is no comprehensive experimental study that jointly evaluates classical and learning-based modularity optimization methods under a unified framework. This paper addresses this gap with the following:

Contributions

- We present the first large-scale experimental comparison of classical and learning-based modularity optimization clustering algorithms, with a systematic review of 17 representative methods, spanning greedy, multilevel, metaheuristic, streaming, exact, and learning-based approaches.
- We release a unified open-source experimental framework that adapts both classical and learning-based methods behind a common interface, enabling reproducible benchmarking and future research.
- We propose a taxonomy of modularity-based clustering algorithms, organized by optimization paradigm and algorithmic strategy.

2 Preliminary

Graphs and clusterings. An (*attributed*) graph is defined as $G = (V, E, \mathcal{F})$, where $V = \{v_1, v_2, \dots, v_n\}$ is a set of n vertices, $E \subseteq V \times V$ is the set of undirected edges, and $\mathcal{F} = \{f_1, f_2, \dots, f_n\}$ denotes node attributes, with each $f_i \in \mathbb{R}^x$ an attribute vector associated with vertex v_i , and x the attribute dimension. We write $n := |V|$ and $m := |E|$ for the number of vertices and edges, respectively. For a vertex $u \in V$, its *neighborhood* is $N(u) := \{v \in V : \{u, v\} \in E\}$, and its *degree* is $\deg(u) := |N(u)|$. Let $A \in \{0, 1\}^{n \times n}$ denote the adjacency matrix of G , where $A_{ij} = 1$ if $\{v_i, v_j\} \in E$ and $A_{ij} = 0$ otherwise. Let $\mathbf{d} \in \mathbb{R}^n$ denote the degree vector with $\mathbf{d}_i = \deg(v_i)$.

A *clustering* C of G is a partition of the vertex set V into k disjoint blocks $V_1, \dots, V_k$ such that $V_1 \cup \dots \cup V_k = V$. Classical algorithms operate solely on the graph structure (V, E) , while learning-based methods additionally exploit node attributes $\mathcal{F}$. Learning-based approaches often aim to discover communities that are coherent both in terms of graph connectivity and attribute similarity.

Modularity. The *modularity* score of a clustering is the fraction of edges that are within one cluster minus the expected fraction if the edges were distributed at random. Let $L_i := |\{\{u, v\} \in E : u, v \in V_i\}|$ denote the number of edges internal to block V_i . The modularity of C is defined as

$$Q(C) := \sum_{i=1}^k \left(\frac{L_i}{m} - \left(\sum_{u \in V_i} \frac{\deg(u)}{2m} \right)^2 \right)$$

Equivalently, modularity can be expressed in spectral form. Let $C \in \{0, 1\}^{n \times k}$ denote the (hard) cluster assignment matrix, where $C_{ip} = 1$ if vertex v_i belongs to cluster p and $C_{ip} = 0$ otherwise. Define the *modularity matrix* $B := A - \frac{\mathbf{d}\mathbf{d}^\top}{2m}$. Then modularity can be written as

$$Q(C) = \frac{1}{2m} \text{Tr}(C^\top B C).$$

This spectral formulation serves as the foundation for many continuous and learning-based approaches.

Graph neural networks. Graph neural networks (GNNs) are a class of neural architectures that compute vector representations for vertices in an attributed graph by iteratively aggregating information from local neighborhoods. In the context of community detection, GNNs are used to map vertices into a representation space in which structurally or semantically related vertices are close. Learning-based clustering methods then infer communities indirectly by optimizing objectives such as continuous or differentiable relaxations of modularity, mutual-information maximization, or contrastive criteria defined over these representations or over derived *soft cluster assignments*, which encode the strength of association between nodes and communities.

3 Background and Methods

We review objective functions and algorithmic paradigms for community detection and introduce the taxonomy used in this work.

Many clustering quality measures have been proposed, including *coverage*, *performance* [?], *inter-cluster conductance* [?], *surprise* [?], and *modularity* [?]. Coverage, the fraction of intra-cluster edges, is simple but favors trivial solutions. Modularity mitigates this by comparing observed coverage to its expectation under a null model, and has become one of the most widely used objectives for community detection [?]. It is the primary quality function in large-scale benchmarks such as the DIMACS challenges [?] and is the focus of this work. While Aref et al. [?] report that modularity heuristics rarely attain optimal solutions, Ans'otegui et al. [?] show that the VIECLUS's modularity maximization heuristic routinely computes optimal solutions on small to medium-sized graphs. Alternative objectives capture complementary notions of community structure: *Weighted Community Cluster* (WCC) maximizes the number of triangles within each community [?], implemented in the SCALABLE COMMUNITY DETECTION algorithm [?], and the *Map Equation*, which measures the expected description length of a random walk [?], used in INFOMAP [?], INFOFLOW [?], and the recent learning-based NEUROMAP [?]. We do not include all the methods based on alternative objective functions in order to maintain our focus on modularity maximization.

Classical modularity-based algorithms use discrete optimization heuristics that consider graph topology only. More recently, learning-based approaches to community detection have been proposed, motivated by the development of GNNs that can model node attributes as feature vectors to capture additional graph information beyond topology alone. These methods typically optimize training objectives that are explicitly aligned with modularity or related structural principles. Compared to classical heuristics, learning-based methods trade exact modularity gains for flexibility, the ability to incorporate node attributes, and increased generalization across graphs. Despite extensive work in both paradigms, existing surveys do not provide a unified experimental comparison of classical and learning-based modularity optimization methods. Prior surveys typically focus on either classical heuristics [?] or learning-based methods [????]. Li et al. [?] compare classical and deep learning methods, but limit comparison to Python-based

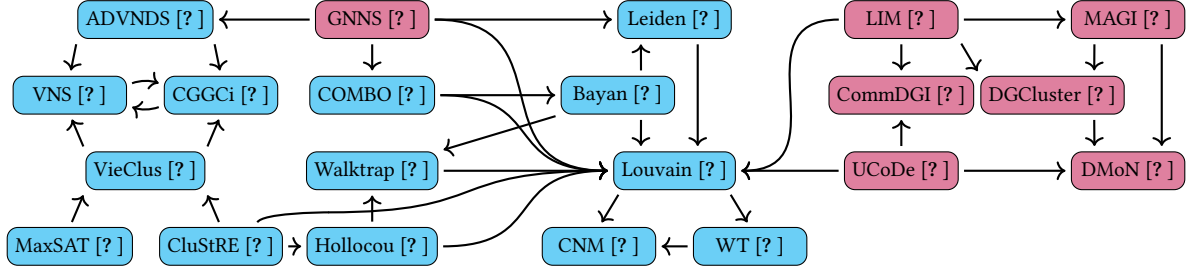


Figure 1: Previous experimental comparisons visualized. Each directed edge indicates that the source algorithm used the destination algorithm in the experimental evaluation. The colors indicate either a learning-based method when red, or a classical optimization algorithm when blue.

Table 1: Comparison of benchmarked methods. Code indicates implementation availability (×: unavailable; ★: available but not ready for use; ★★: open source with well-defined I/O). Features indicate parallelism (P), automatic or user-specified number of clusters (A/K), learning-based methods (L), use of node features (F), and availability in portable Python packages (Py). The bold methods are exact solvers.

Method	Year	Code	Features
CNM [?]	2004	★★ [?]	. A . . Py
WALKTRAP [?]	2005	★★ [?]	. A . . Py
WT [?]	2007	★★ [?]	. A . . Py
LOUVAIN [?]	2008	★★ [?]	. A . . Py
CGGCI [?]	2012	×	. A . . .
VNS [?]	2012	×	. A . . .
COMBO [?]	2014	★★ [?]	. A . . Py
ADVNDs [?]	2017	×	. A . . .
HOLLOCOU [?]	2017	★★ [?]	. A . . .
VIECLUS [?]	2018	★★ [?]	P A . . .
LEIDEN [?]	2019	★★ [?]	. A . . Py
DMoN [?]	2020	★ [?]	P K L F .
COMMdGI [?]	2020	★ [?]	P K L F .
GNNS [?]	2022	★ [?]	P A L . .
UCoDe [?]	2023	★ [?]	P K L F .
DGCLUSTER [?]	2023	★ [?]	P A L F .
MAGI [?]	2024	★ [?]	P K L F .
BAYAN [?]	2024	★★ [?]	. A . . .
CLuStRE [?]	2025	★★ [?]	. A . . .
LIM [?]	2025	★ [?]	P A L F .
MaxSAT [?]	2025	×	. A . . .

solvers, thus lacking a comprehensive comparison of the state-of-the-art. In addition, algorithmic papers in both paradigms rarely evaluate against methods outside their own class (see Figure ??).

This work addresses this gap by providing a comprehensive experimental comparison of modularity-based community detection algorithms under a unified framework. Here, we provide a taxonomy of the considered methods listed in Table ?. We organize methods by *optimization paradigm*, distinguishing between *classical* combinatorial algorithms and *learning-based* approaches with trainable parameters. Classical methods are further grouped by their

dominant optimization mechanism (e.g., greedy, multilevel, or meta-heuristic) and by computational setting (e.g., streaming and exact solvers). Learning-based methods are categorized by how modularity is used during training, separating approaches that directly optimize continuous relaxations of modularity from those that use modularity as a guiding signal within broader learning objectives. Within each subgroup, algorithms are listed chronologically.

3.1 Classical Modularity Optimization Methods

3.1.1 Greedy agglomeration and multilevel local search. Early approaches to modularity maximization focused on greedy agglomeration, merging communities to directly increase modularity. Subsequent work identified limitations of purely agglomerative strategies, notably unbalanced merges and poor scalability, shifting toward multilevel formulations based on local vertex moves.

Clauset–Newman–Moore (CNM) [?] is a hierarchical agglomerative algorithm that greedily maximizes modularity. Starting from singleton communities (each vertex assigned to its own cluster), it repeatedly merges the pair of communities with the largest modularity gain. The algorithm produces a dendrogram representing the full merge hierarchy and returns the clustering attaining maximum modularity. Its running time is $O(md \log n)$, where d is the dendrogram depth.

Wakita–Tsurumi (WT) [?] refines CNM by addressing its tendency to produce highly unbalanced merges that degrade scalability on large graphs. While preserving the same agglomerative structure, WT modifies merge selection to account for both modularity gain and a community-size ratio, biasing the process toward more balanced agglomerations and improving scalability.

LOUVAIN [?] replaces community-pair merging with greedy local vertex moves. Vertices are iteratively moved to neighboring communities whenever the induced modularity gain is positive. Once no improving move exists, communities are contracted into supernodes and the process is repeated, yielding a multilevel optimization scheme. Each local-moving phase runs in time linear in the number of edges, and the number of multilevel passes is typically small, resulting in near-linear runtime in practice.

LEIDEN [?] addresses LOUVAIN’s tendency to produce disconnected communities by introducing a refinement phase before contraction. This phase splits and restructures communities through probabilistic, non-greedy merges, while guaranteeing that resulting communities are connected and locally optimal.

3.1.2 Non-modularity baselines. We include two widely used non-modularity baselines which rely on random walks to model how information or flow propagates through the network, providing an alternative perspective to modularity optimization.

WALKTRAP [?] performs hierarchical clustering based on the intuition that random walks on a graph tend to get "trapped" into densely connected regions corresponding to communities. Each vertex is characterized by the probability distribution over vertices reached by a random walk starting from it, and a structural similarity distance between vertices (and communities) is defined by comparing these distributions. Communities are then obtained by agglomerative merging of the closest pairs under this distance, producing a dendrogram similar to CNM. The algorithm runs in $O(mnd)$ time, where d is the depth of the resulting dendrogram.

INFOMAP [?] formulates community detection as an information-theoretic problem, seeking a clustering that minimizes the expected description length of a random walk under the map equation. It uses a LOUVAIN inspired greedy multilevel search, augmented with fine- and coarse-tuning steps to escape local minima like LEIDEN.

3.1.3 Metaheuristics and ensemble/hybrid search. Beyond greedy and multilevel heuristics, several methods apply metaheuristic or ensemble-learning strategies to modularity maximization. Representative examples include Variable Neighborhood Search (VNS) [?], which systematically perturbs the current clustering to escape local optima, and ensemble-based approaches such as CGGC1-RG [?], which stabilize modularity optimization by consolidating recurring community structures across multiple runs. These methods combine diversification mechanisms (e.g., randomized perturbations or multiple initializations) with intensification via modularity-based local refinement. While more computationally expensive than purely greedy algorithms, such strategies often achieve higher modularity by exploring a larger search space.

COMBO [?] considers pairs of communities and evaluates full re-distributions of vertices between them using Kernighan–Lin–style refinements, instead of relocating individual vertices as in LOUVAIN. By unifying merges, splits, and partial recombinations, COMBO can escape local optima that trap multilevel greedy methods, at the cost of higher computational complexity. The algorithm runs in $O(n^2C)$ time, where C is the number of clusters.

VIECLUS [?] further extends ensemble-based optimization by adopting a memetic (evolutionary) framework. Rather than refining a single partition, it maintains a population of clusterings and generates new solutions through recombination and mutation. Recombination exploits agreement between high-quality solutions via overlay clustering, where two vertices are grouped together only if they co-occur in both parent solutions, while mutation introduces controlled perturbations to preserve diversity. Each offspring is refined using LOUVAIN or multilevel local search and retained only if it improves modularity. After a set time limit, VIECLUS returns the best clustering found thus far.

3.1.4 Streaming algorithms. Most classical modularity algorithms assume random access to the full graph, which becomes infeasible for large or dynamically generated networks. Streaming algorithms address this limitation by processing the graph incrementally under strict memory constraints, typically allowing only a single or few passes over the data.

HOLLOCOU [?] is a one-pass edge-streaming algorithm. The algorithm starts from the singleton clustering and processes edges sequentially. For an arriving edge (u, v) , HOLLOCOU decides whether to merge the communities of u and v based on their current community volumes. If both volumes are below a fixed threshold, the node belonging to the smaller-volume community joins the other. The authors show that under random stream orderings, this local, volume-based merging rule corresponds to a greedy optimization of modularity in the streaming setting.

CLUSTRE [?] follows the node-streaming model unlike the edge-streaming model of HOLLOCOU. Here, nodes arrive sequentially together with their neighborhood lists. Starting from singleton clustering, each node is assigned to the cluster among its neighbors that maximizes the modularity gain similar to a LOUVAIN local move. During streaming, a quotient graph is constructed whose nodes represent clusters and whose weighted edges summarize inter- and intra-cluster connectivity, preserving modularity exactly. This quotient graph is optionally refined using VIECLUS. The graph may be re-streamed to perform further improvements until convergence.

3.1.5 Exact solvers. In contrast to heuristic approaches, exact algorithms provably compute globally optimal modularity clusters. Since modularity maximization is NP-hard [?], these methods rely on exponential-time search augmented with bounding techniques or logical formulations such as weighted partial MaxSAT [?], and are therefore limited to small or medium-sized graphs.

BAYAN [?] is an exact branch-and-cut algorithm for modularity maximization. It formulates modularity as an integer program with binary co-membership variables and enforces consistency via triangular constraints on node triples, which branch on whether three nodes must belong to the same community or cannot all be clustered together. Upper bounds are obtained by solving linear programming relaxations, while lower bounds are computed from feasible clusterings produced by COMBO. Branching decisions either contract node triples into supernodes or discourage their co-assignment, and the algorithm terminates once optimality is certified or a prescribed optimality gap is reached.

3.2 Learning-Based Methods

3.2.1 Continuous and differentiable modularity maximization. These approaches replace discrete community assignments with continuous representations, often learned using GNNs, expressing modularity as a continuous or differentiable objective optimized over node representations or soft cluster assignments.

DMoN [?] introduces a differentiable relaxation of modularity and realizes it using a GNN. A GNN followed by a softmax function produces soft cluster assignments, which are optimized using an objective whose main term corresponds to modularity in spectral form. The objective further includes a *collapse regularization* term that prevents trivial solutions in which all nodes are assigned to the same cluster.

GNNs [?] formulates modularity maximization as a continuous optimization process over soft cluster assignments. Starting from random assignments, node assignments are iteratively updated by combining previous assignments with modularity-weighted aggregation of neighboring assignments. Activation functions and

normalization steps amplify valid assignments supported by positive modularity contributions. The procedure is repeated for multiple random initializations and parameter choices, and the best converged solution is discretized to obtain the final clustering.

Most learning-based modularity methods, including DMoN, require the number of communities to be specified a priori. **DGCLUSTER** [?] eliminates this requirement by learning node embeddings with a GNN and clustering implicitly through a joint optimization objective. Node embeddings learned by a GNN induce soft pairwise community assignments, which parameterize a differentiable modularity objective. A secondary loss incorporates auxiliary local information, such as partial labels or clusterings from classical heuristics, by aligning embedding similarities with a pairwise similarity matrix. After end-to-end optimization, final communities are obtained by clustering the learned embeddings using BIRCH [?].

While DGCLUSTER removes the need to predefine the number of communities, it relies on jointly optimizing multiple objectives which increases model and training complexity. **LIM** [?] addresses this trade-off with a single-objective formulation that still does not require prior specification of the number of communities. The method first applies LOUVAIN to extract coarse global communities, which are summarized as structural community centers in a feature space. A graph convolutional network then learns node representations that capture local structural information and node attributes. Soft cluster assignments are computed by measuring similarity between node representations and the structural centers, and a single modularity objective defined over these assignments is optimized to obtain the final clustering.

3.2.2 Contrastive/infomax objectives. These methods do not optimize modularity directly, but instead use it to guide representation learning through alternative objectives.

Community Deep Graph Infomax (COMMDGI) [?] learns community-aware node representations by jointly maximizing mutual information between nodes and the graph and between nodes and their communities. Unlike other methods that directly optimize modularity, modularity is used here as an auxiliary objective together with mutual information, via a soft k -means-style clustering layer, to encourage structurally coherent communities.

Unified Community Detection (UCoDe) [?] optimizes a single contrastive objective derived from the modularity matrix, encouraging high similarity within communities and low similarity across communities. This contrastive formulation allows UCoDe to recover non-overlapping communities while naturally supporting overlap, without requiring separate objectives or post-processing. Compared to combined-objective methods such as COMMDGI, UCoDe uses a single, interpretable loss and requires minimal hyperparameter tuning.

MAGI [?] establishes a formal connection between modularity maximization and graph contrastive learning by showing that modularity naturally defines positive and negative node pairs via the sign of the modularity matrix. Specifically, node pairs with positive modularity coefficients are treated as positive samples, while pairs with non-positive coefficients are treated as negatives, yielding a contrastive objective equivalent to relaxed modularity maximization. A GNN encoder is trained using a community-aware

contrastive loss, with mini-batch modularity matrices constructed via random-walk sampling for scalability.

4 Evaluation Framework

In this section, we describe the benchmarking methodology used to evaluate modularity-based graph clustering algorithms under controlled and reproducible conditions. A central contribution is the integration of 17 clustering algorithms behind a unified benchmarking interface, offering a fair comparison across diverse implementations. In addition to enabling the experiments presented in this survey, it will also serve as an openly available resource for future work on graph clustering, simplifying the evaluation of older methods on new data or machines. This section also includes details on the datasets we used to evaluate the clustering algorithms.

4.1 Benchmarking Interface

As shown in Table ??, over half of the clustering algorithms in the literature lack ready-to-use interfaces for clustering new data. Furthermore, among those with properly documented code, there are wide differences in the input graph format, output formats, and the additional information they report, such as time and memory consumption. This lack of standardization complicates fair benchmarking, as differences in preprocessing, reporting, and resource accounting can substantially affect observed performance.

To address this problem, we aim to provide an interface with a single common input and output format and consistent time and memory monitoring. The interface exposes a common set of inputs, while allowing solver-specific parameters to be provided when applicable:

$$\text{CLUSTER}(G, T_{\max}, p, k, M_{\max}, \mathcal{F})$$

where G is the input graph, $T_{\max}$ is the wall-clock time limit, p is the number of threads available to the algorithm, k is the number of clusters to compute, $M_{\max}$ is the memory limit in gigabytes, and $\mathcal{F}$ are the graph features. Parameters p , k , and $\mathcal{F}$ are only used by a subset of the solvers. From Table ??, parallel solvers use p , those that compute a fixed number of clusters use k , and any solver that uses graph features can optionally accept $\mathcal{F}$. When $\mathcal{F}$ is not provided, but the solver expects features, a single 1.0 value is assigned to each vertex as the feature. This interface is designed to capture the most common operational requirements across clustering algorithms while enforcing consistent resource limits and output reporting.

Whenever possible, we use the authors' implementations or the most commonly used library implementation. The exact setup for each algorithm is described in our openly available GitHub repository¹. We make only minimal, deliberate modifications where necessary to ensure consistent timing, memory accounting, and output quality measures based on the following rules:

- Parsing the input and writing the output to a file is not counted in the timing of the algorithm.
- For search-based algorithms that always run for the entire $T_{\max}$ duration, the recorded computation time is defined as the point in time when the best solution is found.

¹<https://github.com/ClusterBenchmark/ClusterBenchmark>

- Learning-based methods that train a neural network as part of the algorithm will break the training loop at $T_{\max}$, and use the current model at that time to produce the final clustering.

4.2 Parameter Settings

Where possible, we use the default configurations for all algorithms. Here, we detail the exact parameters chosen for each algorithm. Algorithms that are not listed do not expose any parameters.

Parameter selection policy. To reflect how algorithms are typically used in practice, we deliberately avoid any dataset-specific or instance-specific parameter tuning. Our goal is to benchmark available implementations as they are, rather than hypothetical optimal configurations that would require extensive tuning effort.

- WALKTRAP: walk length = 4.
- LOUVAIN: resolution = 1.
- HOLLOCOU: $v_{\max} = 10\,000$.
- VIECLUS: time limit = 1 hour.
- LEIDEN: resolution = 1, $\beta = 0.02$, iterations = 2.
- DMOON: 1 hidden layer (64), 1 000 epochs, lr = 0.001.
- COMMDGI: 1 hidden layer (512), 1 000 epochs, lr = 0.001.
- GNNS: GNNS100 variant.
- UCODE: 2 hidden layers (256), 1 000 epochs, lr = 0.001.
- DGCLUSTER: hidden layers (256, 128), 300 epochs, lr = 0.001.
- MAGI: hidden layers (512, 256), 100 epochs, lr = 0.01.
- BAYAN: Gurobi ILP solver [?], gap = 10^{-4} , resolution = 1.
- CLUSTRE: light mode, modularity objective.
- LIM: 1 hidden layer (512), 1 000 epochs, patience = 200, lr = 0.001.

For stochastic algorithms with an explicitly specified seed, we use the run ID as the seed for each of the five runs. While many of these algorithms include multiple configurations, we use either the configuration recommended by the authors or the default configuration provided with the implementation.

4.3 Dataset

We consider three datasets commonly used in the literature. The first set consists of 10 real-world graphs with known community labels; we refer to this dataset as the GROUND-TRUTH graphs. The machine learning algorithms commonly use these graphs, and the details of the instances are shown in Table ???. Next, we consider the instances used in the 10th DIMACS implementation challenge on graph clustering [?]; we call this dataset DIMACS graphs. This dataset is popular among the classical modularity optimization methods, and the details of these graphs are shown in Appendix ??. Finally, we use synthetically generated graphs with node features generated using the attributed, degree-corrected stochastic block model (ADC-SBM) [?]. This generator plants k clusters in the graph, and generates random edges via a distribution that is conditional on those clusters. It does so using the two input parameters d and $d_{\text{out}} \leq d$, where d is the expected degree for each vertex and d_{out} is the expected number of neighbors for a node in a different cluster than its own. The ADC-SBM generator also produces node features that are correlated with the community structure. It does so using the two input parameters σ for intra-cluster center variance and σ_c for cluster center variance.

Table 2: GROUND-TRUTH graphs. The $|C|$ column shows the number of clusters and x the number of features.

Instance	n	m	x	$ C $
Cora	2 708	5 278	1 433	7
Citeseer	3 327	4 552	3 703	6
Amazon-Photo	7 650	119 081	745	8
Amazon-Computers	13 752	245 861	767	10
Coauthor-CS	18 333	81 894	6 805	15
Pubmed	19 717	44 324	500	3
Coauthor-Physics	34 493	247 962	8 415	5
ogbn-Arxiv	169 343	1 157 799	128	40
Reddit	232 965	57 307 946	602	41
ogbn-Products	2 449 029	61 859 012	100	47

5 Experiments

In this section, we present benchmark results obtained using the unified benchmarking interface described in Section ??. We report results on real-world graphs with and without ground-truth labels, as well as on synthetically generated graphs with controlled structural and feature noise. Rather than exhaustively ranking algorithms, these experiments illustrate how different algorithmic design choices behave under a standardized benchmarking setup. To support this, we also provide a dynamic website reporting updated results².

5.1 Computational Environment

All algorithms are evaluated on the same machine to eliminate variability due to hardware differences and ensure fair comparison. While some methods could potentially benefit from specialized accelerators such as GPUs, we intentionally restrict all experiments to a shared CPU-based environment to maintain consistency across algorithms. We use a machine with an AMD EPYC 9754 128-core processor with 755 GB of memory, running Ubuntu 24.04.3 with Linux kernel 6.8.0-87. For algorithms with compiled software, we use GCC version 13.3.0, and for Python implementations we use Python version 3.12.3.

To accelerate the overall benchmarking process, we evaluate up to three algorithm instances concurrently on the same machine. This level of concurrency is deliberately conservative relative to the available hardware: the system provides 128 CPU cores and 755 GB of memory, ensuring substantial resource headroom for each run and minimizing contention effects. In particular, we observed that a single process cannot saturate memory bandwidth on this platform, making cross-talk between concurrently evaluated algorithms negligible in practice. Algorithms that support parallel execution internally are evaluated with 16 threads. We also record results using a single thread to test scalability. Each algorithm–instance pair is subject to a wall-clock time limit of one hour and a memory limit of 250 GB. Runs exceeding the memory limit are terminated and recorded as *memory limit exceeded* (MLE). Time limits are enforced more leniently, as several learning-based methods train for a fixed number of iterations rather than converging to a stopping criterion; only runs exceeding the time limit by more than 50% are terminated and recorded as *time limit exceeded* (TLE).

²<https://clusterbenchmark.github.io/>

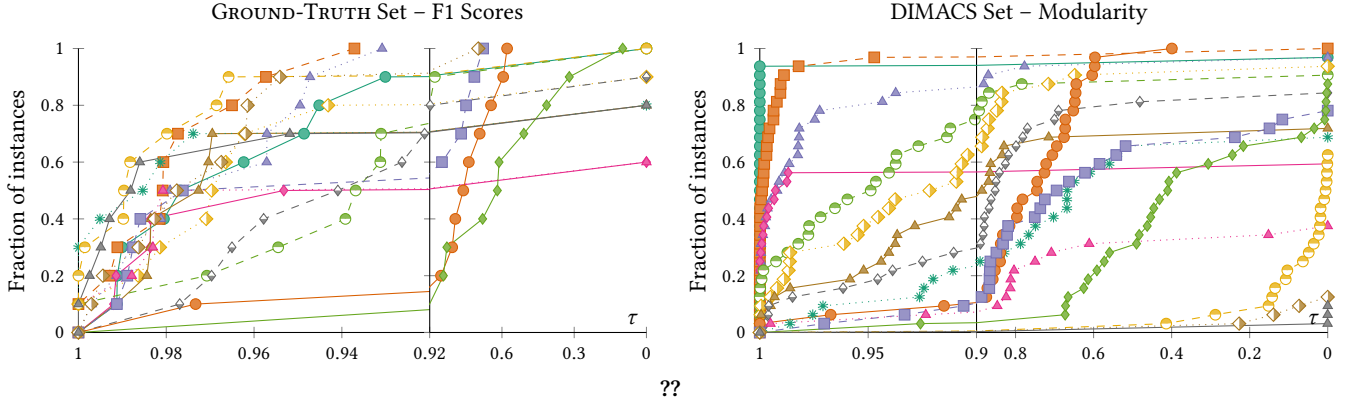


Figure 2: Performance profiles for our first two datasets. We use F1 scores for the GROUND-TRUTH set and modularity for the DIMACS instances. For clarity, we split the x-axis to highlight the differences between the best algorithms near one.

For algorithms requiring a prescribed number of clusters, we provide the ground-truth value when available and set $k = 16$ otherwise, following common practice. For stochastic methods, we report the best result over five independent runs. In addition to modularity (Section ??), we evaluate clustering quality using Normalized Mutual Information (NMI) and an edge-restricted pairwise F1 score. The NMI score measures the level of agreement between the predicted clusters and the ground-truth clusters. For more detailed information, refer to Danon et al. [?]. The F1 score is computed over adjacent vertex pairs: an edge (u, v) is a true positive (TP) if u and v are assigned to the same cluster by the algorithm as well as in the ground-truth clusters. Similarly, we define false positives (FP), true negatives (TN), and false negatives (FN). The F1 score is then calculated using the formula: $F1 := \frac{2TP}{2TP+FP+FN}$.

Performance profiles. Performance profiles are well-suited for benchmarking multiple algorithms, as they summarize relative performance across instances without requiring direct aggregation of objective values [?]. A performance profile assigns each algorithm a non-decreasing, piecewise-constant function. The y-axis represents the fraction of graphs where the objective value is better than or equal to τ times the best observed objective value from any algorithm. Reading a performance profile, an algorithm performs well if it achieves lower τ values for higher fractions of instances.

5.2 State-of-the-Art Comparisons

Figure ?? shows the main results of the experimental comparison. Across both datasets, VIECLUS achieves the highest modularity whenever it completes within the imposed resource limits, consistent with prior findings [?]. As expected for a memetic search-based method, it exceeds the memory limit on the two largest DIMACS instances.

On the GROUND-TRUTH dataset, many methods achieve similarly high F1 scores. Among them, MAGI achieves the strongest and most stable performance. This highlights a general mismatch between modularity optimization and externally defined communities. For example, the Citeseer ground-truth clustering has a modularity of 0.54, whereas all methods produce clusterings with modularity

above 0.54. This discrepancy reflects the fact that citation behavior does not necessarily align with categorical labels, as Figure ?? also indicates.

For the DIMACS graphs, differences between methods are more pronounced, with modularity-focused algorithms clearly outperforming learning-based approaches. Among learning-based methods, performance varies substantially depending on whether the method is designed to operate without node features, with GNNS performing best in this setting.

The streaming methods HOLLOCOU and CLUSTRE trade solution quality for extreme efficiency. While their F1 scores are lower on the GROUND-TRUTH dataset, CLUSTRE achieves competitive modularity on DIMACS instances while using orders of magnitude less time and memory. The exact algorithm BAYAN terminates only on the eight smallest instances from the DIMACS dataset and is therefore excluded from the performance profiles. For these graphs, VIECLUS found the optimal clustering for all of them, LEIDEN for 7/8, GNNS for 5/8, COMBO for 4/8, and LOUVAIN for 3/8. The remaining algorithms did not find any known optimal solutions.

5.3 Quality–Efficiency Tradeoffs

Figure ?? summarizes the tradeoff between clustering quality and execution time using geometric means across instances. Streaming methods occupy the extreme low-latency regime, while greedy modularity methods (LOUVAIN, LEIDEN) achieve a favorable balance between quality and efficiency. The search-based algorithm VIECLUS achieves the highest quality, but at a substantially higher computational cost. Similarly, learning-based methods generally attain high solution qualities at substantially higher computational cost.

Peak memory usage varies significantly across methods, and detailed measurements for the GROUND-TRUTH dataset are presented in Appendix ?. This appendix also offers insights into the number of training iterations completed by each learning-based algorithm. Although the largest instances often face memory or time constraints, most algorithms complete all training iterations, showing that GPUs or other accelerators would not have improved the results for those smaller graphs. We also evaluate the parallel algorithms sequentially to analyze how additional computing resources affect their performance; detailed measurements are presented in Appendix ?.

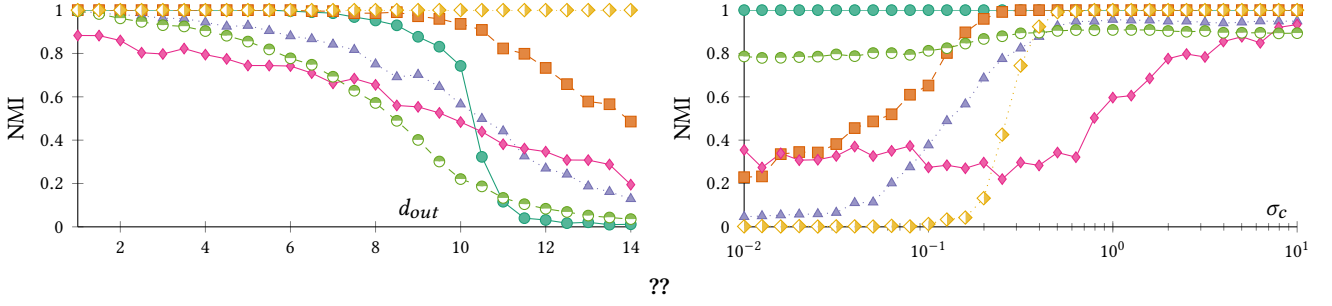


Figure 3: NMI scores on graphs generated with the ADC-SBM graph generator. All graphs have $n = 1\,000$ vertices, $k = 4$ clusters, $d = 20$ average degree, and $\sigma = 1$ intra-cluster center variance. The left plot varies d_{out} while keeping $\sigma_c = 3$, and the right plot varies σ_c while keeping $d_{out} = 2$.

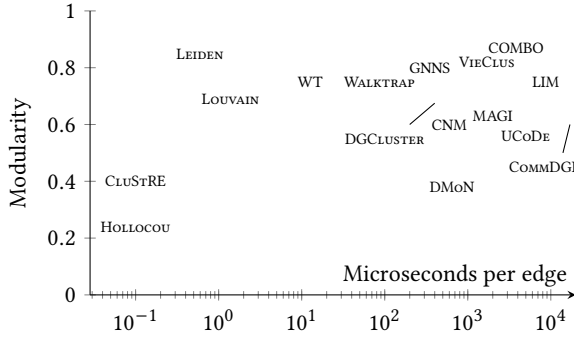


Figure 4: Scatter plot showing the ratio between running time and solution quality, using the geometric mean for both measures on the GROUND-TRUTH graphs. To normalize the running time per graph, we divide it by the number of edges, yielding a time-per-edge measure.

5.4 Synthetic Graphs

To further evaluate the impact and sensitivity of graph features across clustering algorithms, we include experiments with syntactically generated graphs with vertex features. These experiments illustrate how feature-aware methods respond to controlled degradation of structural and attribute signals, rather than ranking the methods. For reference, we also include VIECLUS, which performed best among the pure modularity-optimizing methods. We conduct two sets of experiments with the ADC-SBM generator. First, we generate a sequence of graphs with gradually higher d_{out} , causing the graph structure to carry less information about the original k clusters. And second, we generate graphs with strong community structure as we gradually increase the cluster center variance σ_c , leading the graph features to carry less information about the clusters.

Figure ?? summarizes the results of the two ADC-SBM experiments. When the feature signal is strong, MAGI and DMOON achieve the highest NMI scores, and beyond $d_{out} \approx 11$, all feature-aware methods outperform VIECLUS. The results also highlight clear differences in how methods balance structural and feature information. MAGI relies primarily on node features, as evidenced by its strong performance under structural noise and its sensitivity to feature degradation. In contrast, DMOON remains robust in both settings, indicating a more balanced use of topology and attributes. COMM DGI exhibits similar trends but performs consistently worse than

DMOON. DGCLUSTER shows the opposite bias to MAGI, favoring graph structure over features, while UCoDe displays a similar but less pronounced tendency. We exclude LIM from this figure, as it consistently collapsed to a single cluster, yielding zero NMI across all configurations.

6 Conclusion

We surveyed 17 modularity-based graph clustering algorithms, covering classical heuristics, parallel methods, and recent learning-based approaches. When modularity maximization is the sole objective, the results are clear: VIECLUS consistently achieves the highest modularity values when sufficient memory is available. LEIDEN, an improved variant of LOUVAIN, closely follows in solution quality and offers substantially faster runtimes. This trade-off reflects their optimization strategies: VIECLUS relies on time-budgeted search, while LEIDEN uses a greedy refinement scheme.

Beyond modularity alone, other algorithms exhibit their own unique strengths. Streaming methods such as CLUSTRE offer extremely low runtimes and memory footprints, making them well-suited for large-scale or resource-constrained settings. Learning-based approaches, including GNNS, demonstrate that neural methods are approaching the performance of hand-crafted heuristics. However, they still require substantially more memory and do not yet achieve competitive modularity scores. At the same time, there is growing interest in expanding modularity optimization with auxiliary objectives to recover ground-truth communities better. In this setting, feature-aware methods can be advantageous: MAGI achieves the strongest ground-truth recovery in our experiments, while DMOON remains effective when community structure is encoded between topology and node attributes.

To support future work in this area, we introduce a unified benchmarking framework that enables direct, reproducible comparisons across a wide range of modularity-based clustering algorithms. We hope that this framework lowers the barrier for evaluating new methods against established baselines and encourages more systematic experimental studies of graph clustering algorithms.

Acknowledgments

We acknowledge support by DFG grant SCHU 2567/5-1.

Disclosure of Generative AI Use

Generative AI tools were used to assist with language editing and stylistic refinement of the manuscript. All scientific contributions, including algorithm selection, experimental design, implementation, and analysis, were performed by the authors.

A Instances

Table 3: Graph from the 10th DIMACS implementation challenge. There are no features or ground truth clusters for these graphs.

Instance	n	m
karate	34	78
chesapeake	39	170
dolphins	62	159
lesmis	77	254
polbooks	105	441
adjnoun	112	425
football	115	613
jazz	198	2 742
celegansneural	297	2 148
celegans-metabolic	453	2 025
email	1 133	5 451
polblogs	1 490	16 715
netscience	1 589	2 742
power	4 941	6 594
hep-th	8 361	15 751
PGPgiantcompo	10 680	24 316
astro-ph	16 706	121 251
cond-mat	16 726	47 594
as-22july06	22 963	48 436
cond-mat-2003	31 163	120 029
cond-mat-2005	40 421	175 691
preferentialAttachment	100 000	499 985
smallworld	100 000	499 998
G-n-pin-pout	100 000	501 198
caidaRouterLevel	192 244	609 066
cnr-2000	325 557	2 738 969
eu-2005	862 664	16 138 468
in-2004	1 382 908	13 591 473
road-central	14 081 816	16 933 413
uk-2002	18 520 486	261 787 258
road-usa	23 947 347	28 854 312
uk-2007-05	105 896 555	3 301 876 564

B Memory Usage

Table 4: Memory consumption of the evaluated algorithms on the GROUND-TRUTH dataset. Values are reported in human-readable units. Entries marked TLE indicate runs exceeding the one-hour time limit, while MLE denotes runs exceeding the 250 GB memory limit.

Method	Cora	Citeseer	Amz-Photo	Amz-Comp	CoAuth-CS	Pubmed	CoAuth-Phys	ogbn-Arxiv	Reddit	ogbn-Prod
VIECLUS	22.53 MB	22.53 MB	64.91 MB	130.24 MB	58.7 MB	44.94 MB	134.75 MB	497.74 MB	9.6 GB	TLE
LEIDEN	20.48 MB	20.48 MB	38.91 MB	59.39 MB	34.82 MB	26.62 MB	61.44 MB	211.36 MB	9.01 GB	9.83 GB
LOUVAIN	20.48 MB	20.48 MB	38.91 MB	59.39 MB	32.77 MB	28.67 MB	61.44 MB	210.32 MB	9.01 GB	9.83 GB
COMBO	255.58 MB	314.59 MB	1.09 GB	3.17 GB	5.42 GB	TLE	TLE	MLE	MLE	MLE
GNNS	1.35 GB	1.75 GB	5.49 GB	8.1 GB	8.28 GB	8.31 GB	8.11 GB	6.09 GB	26.37 GB	MLE
WT	20.48 MB	20.48 MB	38.91 MB	59.39 MB	34.82 MB	28.67 MB	61.44 MB	211.36 MB	TLE	TLE
CNM	192.51 MB	179.96 MB	253.07 MB	372.06 MB	234.14 MB	210.71 MB	398.65 MB	TLE	TLE	TLE
WALKTRAP	27.32 MB	20.48 MB	418.48 MB	1.47 GB	935.16 MB	488.84 MB	4.36 GB	99.06 GB	MLE	MLE
COMM DGI	735.03 MB	857.03 MB	2.05 GB	5.34 GB	9.54 GB	9.82 GB	31.09 GB	MLE	MLE	MLE
CLUSTRE	6.14 MB	6.14 MB	6.14 MB	6.14 MB	6.14 MB	6.14 MB	6.14 MB	8.19 MB	8.19 MB	22.53 MB
DMoN	664.01 MB	863.31 MB	778 MB	987.44 MB	3.5 GB	809.3 MB	7.39 GB	735.92 MB	67 GB	77.26 GB
LIM	674.08 MB	741.17 MB	1.92 GB	4.67 GB	7.45 GB	TLE	TLE	MLE	MLE	MLE
HOLLOCOU	2.05 MB	2.05 MB	4.1 MB	6.14 MB	4.1 MB	4.1 MB	8.19 MB	32.39 MB	912.7 MB	1.15 GB
MAGI	807.82 MB	992.96 MB	1.55 GB	2.56 GB	6.15 GB	2.22 GB	15.66 GB	TLE	50.74 GB	MLE
DGCLUSTER	548.86 MB	917.5 MB	879.83 MB	1.39 GB	5.84 GB	1.07 GB	13.2 GB	624.64 MB	150.63 GB	TLE
UCoDe	576.89 MB	771.56 MB	1.7 GB	4.32 GB	7.69 GB	6.94 GB	24.23 GB	MLE	MLE	MLE

Table 5: Number of finished training iterations of the evaluated algorithms on the GROUND-TRUTH dataset. Entries marked TLE indicate runs exceeding the one-hour time limit, while MLE denotes runs exceeding the 250 GB memory limit.

Method	Cora	Citeseer	Amz-Photo	Amz-Comp	CoAuth-CS	Pubmed	CoAuth-Phys	ogbn-Arxiv	Reddit	ogbn-Prod
COMM DGI	1 000/1 000	1 000/1 000	1 000/1 000	1 000/1 000	628/1 000	708/1 000	224/1 000	MLE	MLE	MLE
DMoN	1 000/1 000	1 000/1 000	1 000/1 000	1 000/1 000	1 000/1 000	1 000/1 000	1 000/1 000	1 000/1 000	24/1 000	17/1 000
LIM	1 000/1 000	1 000/1 000	1 000/1 000	874/1 000	552/1 000	TLE	TLE	MLE	MLE	MLE
MAGI	100/100	100/100	100/100	100/100	100/100	100/100	31/100	TLE	18/100	MLE
DGCLUSTER	300/300	300/300	300/300	300/300	300/300	300/300	300/300	300/300	34/300	TLE
UCoDe	1 000/1 000	1 000/1 000	1 000/1 000	1 000/1 000	1 000/1 000	1 000/1 000	450/1 000	MLE	MLE	MLE

C Parallel-Sequential Comparison

Table 6: Parallel and sequential running times for six instances from the GROUND-TRUTH dataset. Only the parallel methods are included in the table. The t_1 column gives the sequential running time in seconds, while the t_{16} column displays the running time with 16 threads.

Method	Cora		Citeseer		Amz-Photo		Amz-Comp		CoAuth-CS		CoAuth-Phys	
	t_1	t_{16}	t_1	t_{16}	t_1	t_{16}	t_1	t_{16}	t_1	t_{16}	t_1	t_{16}
VIECLUS	276.55	83.31	8.29	5.19	93.29	16.25	2 804.96	2 476.70	2 811.52	2 897.85	3 511.06	3 562.30
GNNS	36.40	9.41	48.44	11.90	231.92	43.46	592.24	115.08	668.79	162.44	1 906.75	363.04
COMM DGI	936.72	140.64	2 046.85	288.34	3 606.69	1 284.26	3 616.27	3 416.34	3 613.43	3 602.34	3 664.35	3 610.59
DMoN	55.60	42.19	100.31	50.01	312.63	184.55	658.25	352.90	868.59	212.61	2 060.92	498.45
LIM	811.90	97.63	1 218.18	185.00	3 605.01	618.07	3 614.08	1 909.42	3 633.77	2 130.90	TLE	TLE
MAGI	119.29	56.88	215.38	66.44	1 445.31	535.17	3 605.81	1 572.78	3 635.58	3 414.73	3 691.43	3 614.21
DGCLUSTER	44.78	17.40	91.01	23.75	301.28	98.98	594.48	193.56	833.56	140.19	1 961.84	307.94
UCoDe	223.57	32.04	460.72	62.13	1 305.44	298.82	3 603.07	1 136.63	3 607.82	2 287.33	3 625.18	3 607.02

Temporary page!

$\LaTeX$ was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because $\LaTeX$ now knows how many pages to expect for this document.